

(72 points possible)

✓ Problem 1: Basic functions (19 pts [6, 7, 2, 4])

a, 6 pts) Write a function called `remove_duplicates(L)` that takes as input a list and returns a list where all the duplicates have been removed.

```
#TODO
def remove_duplicates(lst):
    result=[]
    for i in lst:
        if i not in result:
            result.append(i)
    return result
```

```
print(remove_duplicates(['a', 'b', 'c', 'b'])) # should return ['a', 'b', 'c']
print(remove_duplicates(['a','a', 'b','a', 'c', 'b'])) # should return ['a', 'b', 'c']
print(remove_duplicates(['ab', 'ba', 'ab', 'ba'])) # should return ['ab', 'ba']
```

c, 6 pts) Write a function called `largest_revenue_and_quantity(tuples)` that takes as input a list of tuples of (quantity, price) and returns the tuple with the largest revenue of quantity*price. If there are multiple tuples tied for best, then you should return the tied tuple with the largest quantity. If the tuples input list is empty, then return None.

```
#TODO
def largest_revenue_and_quantity(tplLst):
    if not tplLst:
        return None
    topTup = tplLst[0]
    for r,q in tplLst:
        rev = r*q
        if rev>=topTup[0]*topTup[1]:
            topTup = (r,q)
    return topTup
```



```
print(largest_revenue_and_quantity([])) # Should return None
print(largest_revenue_and_quantity([(1, 100), (2, 100), (4, 50)])) # Should return (2, 100)
print(largest_revenue_and_quantity([(2, 3), (4, 3), (3, 4), (2, 1)])) # Should return (3, 4)
```

```
None
(4, 50)
(3, 4)
```

d, 2 pts) Write a function called `factorial(k)` which returns $k! = k*(k-1)*(k-2)*\cdots*2*1$. Note that $0! = 1$.

```
#TODO
def factorial(x):
    fact = 1
    for i in range(1,x+1):
        fact*=i

    return fact
```

```
print(factorial(0)) # Expect 1
print(factorial(3)) # Expect 6
print(factorial(4)) # Expect 24
print(factorial(5)) # Expect 120
```

```
1
6
24
120
```

e, 4 pts) Write a function called `my_exponential(x, k)` which takes as input a float x and an integer n . This function should return

$$\sum_{n=0}^k \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^k}{k!},$$
$$\sum_{n=0}^k \frac{x^n}{n!} = 1 + x + \frac{x^2}{2!} + \cdots + \frac{x^k}{k!},$$

where $k!$ denotes the factorial of k . You must use the factorial function that you coded in Part d. This function is an approximation of the exponential function e^x .

```
#TODO
def my_exponential(x,k):
    expo=0
    for n in range(k+1):
        expo+=(x**n)/(factorial(n))
    return expo
```

```
print(my_exponential(1, 5)) # Expect 2.716666
print(my_exponential(1, 10)) # Expect 2.7182818
print(my_exponential(1, 15)) # Expect 2.71828182845, as k becomes larger
```

```
2.7166666666666663
2.7182818011463845
2.718281828458995
```

Problem 2: Sets and dictionaries (32 pts [4, 8, 4, 2, 14])

a, 4 pts) Write a function `list_difference(L1, L2)` that takes as input two lists $L1$ and $L2$ and returns a set of the elements of $L1$ that do not belong to $L2$. Hint: Make use of the `set difference()` method.

```
#TODO
def list_difference(L1,L2):

    S1,S2=set(L1),set(L2)
    return S1-S2
```

```
print(list_difference([1, 2, 3, 4], [3, 4, 5, 6])) # Expect {1, 2}
print(list_difference([1, 1, 2, 2, 3, 3], [1, 3])) # Expect {2}
print(list_difference([], [])) # Expect set()
```

```
{1, 2}
{2}
set()
```

b, 8 pts) Write a function `names_and_numbers(tuples)` which takes as input a list of tuples where each tuple contains a name (e.g., 'bob') and telephone number (e.g., '123-456-7890'). You should return a dictionary where the keys are the name and the value is a list of telephone number(s). If a name in the tuples list has multiple numbers, the list should contain all of these numbers. If the number is a duplicate, it should not be added to the list.

```
#TODO
def names_and_numbers(tuples):
    retset = set(tuples)
    retDict = {}

    for n,t in retset:
        if(n in retDict.keys()):
            retDict[n].append(t)
        else:
            a = {n:[t]}
            retDict.update(a)
    return retDict
```

```
print(names_and_numbers([('alice', '555-555-5555'), ('bob', '111-222-3333')]))
print(names_and_numbers([('alice', '555-555-5555'), ('bob', '111-222-3333')]))
print(names_and_numbers([('alice', '555-555-5555'), ('bob', '111-222-3333')]))
```

```
<class 'dict_items'>
{'bob': ['111-222-3333'], 'alice': ['555-555-5555']}
<class 'dict_items'>
{'alice': ['555-666-7777', '555-555-5555'], 'bob': ['111-222-3333']}
<class 'dict_items'>
{'bob': ['111-222-3333'], 'alice': ['555-555-5555']}
```

c, 4 pts) Write a function `negate_even_keys(d)` that takes as input a dictionary with integer keys and boolean values. If the key value is even you should negate the boolean value. Return the modified dictionary.

```
#TODO
def negate_even_keys(d):

    for i in d.keys():
        if i%2==0:
            d[i]= not d[i]

    return d
```

```
print(negate_even_keys({1: True, 2: False})) # Expect {1: True, 2: True}
print(negate_even_keys({4: True, 2: False})) # Expect {4: False, 2: True}

{1: True, 2: True}
{4: False, 2: True}
```

d, 2 pts) Write a function `generate_dict(name, hws, tests)` which takes as input a string `name`, a list `hws` of homework scores, and a list `tests` of test scores. The function should return a dictionary of the form:

```
{"name": name, "homework": hws, "test": tests}
```

```
#TODO
def generate_dict(name,hws,tests):
    return {'name':name, 'homework':hws,'test':tests}
```

```
print(generate_dict("scott", [78, 84, 93], [82, 96])) # Expect {'name':
{'name': 'scott', 'homework': [78, 84, 93], 'test': [82, 96]}
```

e, 14) Write a function called `calculate_grades(dictionaries)` which takes a list of dictionaries structured as in Part d as input and outputs a list of dictionaries of the form

```
{"name": name, "grade": grade}
```

where name is the name of the student and grade is a weighted average consisting of **30% of the homework average** and **70% of the test average**. If the "homework" or "test" keys **don't exist**, the **values** used to compute the weighted average **should be 0**. If the key exists with an empty list, the average should also be zero. You may assume there are always name keys.

```
#TODO
def calculated_grades(dictionaries):
    retlist = []
    def avg(list):
        if list==[]:
            return 0
        ret=0
        for i in list:
            ret+=i
        return ret/len(list)

    for name in dictionaries:
        grade = 0
        if('homework' in name.keys()):
            grade+=avg(name["homework"])*0.3
        if('test' in name.keys()):
            grade+=avg(name["test"])*0.7
        retlist.append({'name':name['name'],'grade':grade})

    return retlist
```

```
alice = {"name": "alice", "homework": [100, 100, 100], "test": [100, 100]
bob = {"name": "bob", "homework": [50, 50, 50], "test": [100, 100]}
che = {"name": "che", "homework": [], "test": [30, 50]}
daphne = {"name": "daphne", "test": [30, 50]}
L = [alice, bob, che, daphne]
```

```
print(calculated_grades(L)) # Expect [{'name': 'alice', 'grade': 100.0},
[{'name': 'alice', 'grade': 100.0}, {'name': 'bob', 'grade': 85.0}, {'nam
```

▼ Problem 3: Caesar Cipher (19 pts [3, 3, 6, 6, 1])

The Caesar cipher is a simple encryption method that shifts each letter of the alphabet by a fixed number of positions to create a coded message. A "shift" value is chosen, which determines how many positions each letter is moved. For example, a shift of 3 would mean A becomes D, B becomes E, and Z wraps around to C. In this problem we will build up functions to create our own Caesar Cipher to encrypt and decrypt messages.

a, 3 pts) Write a function `letters_to_ints()` that returns a dictionary that maps the lower case letters in the alphabet to their corresponding integer, i.e., 'a': 1, 'b':2, etc. Hint: Use `strings.ascii_lowercase`.

```
#TODO
import string as strings
def letters_to_ints():
    a = {}
    letters = strings.ascii_lowercase
    curr = 1
    for i in letters:
        new={i:curr}
        a.update(new)
        curr+=1
    return a
```

```
# def letters_to_ints():
```

```
d1 = letters_to_ints()  
print(d1['a']) # Expect 1  
print(d1['z']) # Expect 26
```

```
1  
26
```

b, 3 pts) Write a function `ints_to_letters()` that returns a dictionary that maps the integers to their corresponding letter 1:'a', 2:'b', etc.

```
#TODO  
import string as strings  
def ints_to_letters():  
    a = {}  
    letters = strings.ascii_lowercase  
    curr = 1  
    for i in letters:  
        new={curr:i}  
        a.update(new)  
        curr+=1  
    return a
```

```
d2 = ints_to_letters()  
print(d2[1]) # Expect 'a'  
print(d2[2]) # Expect 'b'
```

```
a  
b
```

c, 6 pts) Write a function called `encrypt(word, shift)` which accepts a string word and integer shift as inputs. This function should shift each letter in word by shift digits and return the encrypted word. Hint: Make use of your helper functions in parts a and b.

```
#TODO
def encrypt(word,shift):
    crypt=""
    for i in word:
        l2i,i2l=letters_to_ints(),ints_to_letters()
        crypt+=i2l[(l2i[i]+shift)%26]
    return crypt
```

```
print(encrypt('abc', 2)) # Expect 'cde'
print(encrypt('hello', 20)) # Expect 'byffi'
```

```
cde
byffi
```

d, 6 pts) Write a function called `decrypt(word, shift)` which accepts a string word that has been encrypted and the integer shift as inputs. This function should return the unencrypted string based on the shift.

```
#TODO
def decrypt(word,shift):
    solved=""
    for i in word:
        l2i,i2l=letters_to_ints(),ints_to_letters()
        if(l2i[i]-shift>0):
            solved+=i2l[(l2i[i]-shift)]
        else:
            solved+=i2l[26+(l2i[i]-shift)]
    return solved
```

```
print(decrypt('cde', 2)) # Expect 'abc'
print(decrypt('byffi', 20)) # Expect 'hello'
```

```
abc
hello
```

e, 2 pts) Decrypt the following `word='hjetgrpaxugpvxaxhixrtmexpaxsdrxdjh'` given that `shift=15`.

```
#TODO
print(decrypt('hjetgrpaxugpvxaxhixrtmexpaxsdrxdjh', 15))

supercalifragilisticexpialidocious
```

✓ AI Statement (2 points)

Please put an X in the box that best matches your AI use for this homework ([X] in the markup will become a checked box). There is no penalty for using AI, but we want to understand how you're using it.

- ☒ I did not use AI to complete this assignment.
- ☐ I didn't use AI to debug or generate code, but I did use it as a Python reference.
- ☐ I didn't use AI to generate code, but I did use it to debug some code.
- ☐ I did use AI to generate some code, but not all the code for any problem.
- ☐ I used AI to solve at least one problem completely.

When you are done, submit both your .ipynb (File->Download->Download .ipynb in Google Colab) and a PDF (File->Print->Save to PDF) to Blackboard where you found this assignment. (The backup PDF helps us give you points if there's a problem with your .ipynb.)

Start coding or generate with AI.

